



포팅메뉴얼

우주드로우 서비스 포팅 매뉴얼 (SSAFY)

1. 문서 개요

1.1 목적

본 문서는 우주드로우 프로젝트를 GitLab 소스 클론 이후 빌드 및 배포할 수 있도록, 필요한 실행 환경, 설정 값, 외부 서비스, 배포 절차를 정리한 포팅 매뉴얼이다.

1.2 대상

- SSAFY 프로젝트 심사자
- 프로젝트 배포 및 운영 담당자

2. 시스템 구성 개요

2.1 아키텍처 구성

```
Client
└─ HTTPS
    │   └─ Nginx (SSL + Reverse Proxy)
    │       │   └─ /api      → Backend (8081)
    │       │   └─ /        → Frontend (80)
    │       │
    │       └─ Backend (Spring Boot)
    │           │   └─ PostgreSQL (AWS RDS)
    │           │   └─ Redis
    │           │   └─ RabbitMQ → AI Service (8000)
    │           │
    │       └─ AI Service (FastAPI)
    │           │   └─ YOLO (객체 탐지)
```

└─ Upstage Solar LLM (감정 해석)
└─ AWS S3 (이미지 저장)

2.2 설계 특징

- RabbitMQ 메시지 큐를 통한 AI 비동기 처리 구조
- Nginx를 통한 SSL 종단 및 Reverse Proxy
- AWS S3를 통한 드로잉 이미지 저장 (Presigned URL 방식)
- YOLO 객체 탐지 + LLM 기반 감정 분석 파이프라인
- 모니터링 스택 분리 (Prometheus + Grafana + Loki)

3. 사용 제품 및 버전

구분	제품	버전	용도
JVM	Eclipse Temurin	17	Backend 실행
WAS	Apache Tomcat	Embedded (Spring Boot)	Spring Boot 내장
Web Server	Nginx	Alpine 최신	Reverse Proxy, SSL, Frontend
Frontend Runtime	Node.js	20 (LTS)	Vite 빌드
Build Tool	Vite	^7.3.1	Frontend 번들링
Framework	React	^19.2.0	Frontend SPA
3D Rendering	Three.js	^0.168.0	별자리/갤럭시 시각화
AI Runtime	Python	3.10	FastAPI 실행
AI Framework	FastAPI	>=0.100.0	AI REST API
Object Detection	Ultralytics (YOLO)	>=8.0.0	드로잉 기호 탐지
Message Queue	RabbitMQ	3-management-alpine	AI 비동기 처리
Cache	Redis	Alpine 최신	세션/토큰 캐시
Database	PostgreSQL	AWS RDS	사용자 데이터
Storage	AWS S3	-	드로잉 이미지
SSL	Let's Encrypt	-	HTTPS 인증서

구분	제품	버전	용도
IDE	IntelliJ IDEA	2023.x	Backend 개발
IDE	VS Code	최신	Frontend / AI 개발

4. 디렉토리 구조

```

S14P21D207/
├── ai/                                # FastAPI AI 서비스 (YOLO + LLM)
├── be/
│   └── spring/                        # Spring Boot Backend
├── fe/                                # React + Three.js Frontend (PWA)
├── infra/
│   ├── docker-compose.yml
│   ├── docker-compose.monitoring.yml
│   ├── .env
│   ├── nginx/
│   │   └── conf.d/
│   │       └── default.conf
│   ├── prometheus/
│   ├── grafana/
│   ├── loki/
│   ├── promtail/
│   └── alertmanager/
└── docs/

```

5. 환경변수 정의

보안상 민감한 값(API Key, DB 계정, AWS 자격증명)은 실제 값 대신 식별자 형태로 기재하였다.

5.1 통합 환경변수 (infra/.env)

```

# Database (AWS RDS PostgreSQL)
DB_URL=jdbc:postgresql://<RDS_ENDPOINT>:5432/WUD
DB_USERNAME=<DB_USERNAME>
DB_PASSWORD=<DB_PASSWORD>

```

```
# RabbitMQ
RABBITMQ_USER=<RABBITMQ_USERNAME>
RABBITMQ_PASS=<RABBITMQ_PASSWORD>

# Redis
REDIS_HOST=redis
REDIS_PORT=6379

# AWS S3
AWS_REGION=ap-northeast-2
AWS_S3_BUCKET=<S3_BUCKET_NAME>
AWS_S3_PREIGNED_URL_EXP_MIN=5
AWS_ACCESS_KEY=<AWS_ACCESS_KEY_ID>
AWS_SECRET_KEY=<AWS_SECRET_ACCESS_KEY>
AWS_S3_PHOTO_PATH=photos

# Domain & SSL
DOMAIN=<EC2_DOMAIN>
CERTBOT_EMAIL=<EMAIL>

# Monitoring
GRAFANA_PASSWORD=<GRAFANA_ADMIN_PASSWORD>
```

5.2 AI Service (ai/.env)

```
AWS_ACCESS_KEY_ID=<AWS_ACCESS_KEY_ID>
AWS_SECRET_ACCESS_KEY=<AWS_SECRET_ACCESS_KEY>
AWS_REGION_NAME=ap-northeast-2
S3_BUCKET_NAME=<S3_BUCKET_NAME>

UPSTAGE_API_KEY=<UPSTAGE_API_KEY>
GMS_KEY=<GMS_API_KEY>
GMS_MODEL=gpt-4o
```

6. Backend 설정 파일

6.1 application.properties

```
spring.application.name=wud_backend
server.port=8081

# Timezone
spring.jpa.properties.hibernate.jdbc.time_zone=Asia/Seoul

# Datasource (PostgreSQL)
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
spring.datasource.driver-class-name=org.postgresql.Driver

# JPA
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.open-in-view=false

# Redis
spring.data.redis.host=${REDIS_HOST:localhost}
spring.data.redis.port=${REDIS_PORT:6379}

# RabbitMQ
spring.rabbitmq.host=rabbitmq
spring.rabbitmq.port=5672
spring.rabbitmq.username=${RABBITMQ_USER}
spring.rabbitmq.password=${RABBITMQ_PASS}

# AWS S3
cloud.aws.region.static=${AWS_REGION}
cloud.aws.s3.bucket=${AWS_S3_BUCKET}
cloud.aws.credentials.access-key=${AWS_ACCESS_KEY}
cloud.aws.credentials.secret-key=${AWS_SECRET_KEY}
```

실제 DB 접속 정보 및 AWS 자격증명은 환경변수를 통해 주입된다.

7. 로컬 실행 가이드

7.1 Frontend

```
cd fe
npm install
npm run dev
```

- 접속: <http://localhost:5173>

7.2 Backend

```
cd be/spring
./gradlew bootRun
```

- 접속: <http://localhost:8081>

7.3 AI Service

```
cd ai
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install -r requirements.txt
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

- 접속: <http://localhost:8000>

7.4 Redis (로컬)

```
docker run -d -p 6379:6379 redis:alpine
```

7.5 RabbitMQ (로컬)

```
docker run -d -p 5672:5672 -p 15672:15672 rabbitmq:3-management-alpine
```

- 관리 UI: <http://localhost:15672> (guest/guest)

8. 서버 배포 가이드

8.1 사전 준비

- AWS EC2 (Ubuntu Linux)
- Docker / Docker Compose 설치
- `infra/.env` 파일 구성
- `ai/.env` 파일 구성
- SSL 인증서 (Let's Encrypt, Certbot 자동 발급)

8.2 배포 실행

```
cd infra
docker compose up -d --build
```

8.3 EC2 최초 배포 단계

1. EC2 접속

```
ssh -i <KEY.pem> ubuntu@<EC2_HOST>
```

2. Docker/Compose 설치

```
# Docker 설치 및 권한 설정 (공식 문서 참고)
docker -v
docker compose version
```

3. 프로젝트 클론

```
git clone <REPO_URL>
cd <REPO_DIR>/infra
```

4. 환경변수 파일 준비

- `infra/.env` 작성
- `ai/.env` 작성
- 필요 값은 본 문서 5장 참조

5. SSL 인증서 발급

- docker-compose 내 certbot 서비스가 자동 발급 및 갱신 처리
- 최초 실행 시 `DOMAIN`, `CERTBOT_EMAIL` 환경변수 확인 필수

6. 컨테이너 빌드/기동

```
docker compose up -d --build
```

7. 서비스 확인

```
docker compose ps
docker compose logs -f backend
# 브라우저: https://<도메인>
```

8.4 Docker Compose 파일 (infra/docker-compose.yml)

```
services:
  nginx:
    image: nginx:alpine
    container_name: nginx
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx/conf.d:/etc/nginx/conf.d:ro
      - /etc/letsencrypt:/etc/letsencrypt:ro
    depends_on:
      - frontend
      - backend
    networks:
      - app-network

  frontend:
    build: ../fe
    restart: unless-stopped
    networks:
      - app-network
```



```
backend:
build: ../be/spring
restart: unless-stopped
env_file:
- .env
environment:
- REDIS_HOST=redis
- REDIS_PORT=6379
depends_on:
redis:
condition: service_healthy
rabbitmq:
condition: service_healthy
networks:
- app-network

redis:
image: redis:alpine
container_name: redis
restart: unless-stopped
healthcheck:
test: ["CMD", "redis-cli", "ping"]
interval: 5s
retries: 5
networks:
- app-network

rabbitmq:
image: rabbitmq:3-management-alpine
container_name: rabbitmq
restart: unless-stopped
env_file:
- .env
environment:
RABBITMQ_DEFAULT_USER: ${RABBITMQ_USER}
RABBITMQ_DEFAULT_PASS: ${RABBITMQ_PASS}
ports:
```

```

- "127.0.0.1:15672:15672"    # SSH 터널링용 관리 UI
healthcheck:
test: ["CMD", "rabbitmq-diagnostics", "ping"]
interval: 10s
retries: 5
networks:
- app-network

ai:
build: ../ai
restart: unless-stopped
env_file:
- ../ai/.env
environment:
- PYTHONUNBUFFERED=1
- RABBITMQ_HOST=rabbitmq
- RABBITMQ_USER=${RABBITMQ_USER}
- RABBITMQ_PASS=${RABBITMQ_PASS}
depends_on:
rabbitmq:
condition: service_healthy
networks:
- app-network

certbot:
image: certbot/certbot
volumes:
- /etc/letsencrypt:/etc/letsencrypt
entrypoint:>
/bin/sh -c "trap exit TERM;
while ;; do
certbot renew;
sleep 12h & wait $$(!);
done"

networks:
app-network:
driver: bridge

```

8.5 Dockerfile (복사/붙여넣기)

be/spring/Dockerfile

```
# Build stage
FROM eclipse-temurin:17-jdk-alpine AS build
WORKDIR /app
COPY . .
RUN chmod +x ./gradlew
RUN ./gradlew clean build -x test --no-daemon

# Runtime stage
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
ENV TZ=Asia/Seoul
COPY --from=build /app/build/libs/*-SNAPSHOT.jar app.jar
EXPOSE 8081
ENTRYPOINT ["java", "-Duser.timezone=Asia/Seoul", "-jar", "app.jar"]
```

ai/Dockerfile

```
FROM python:3.10-slim

ENV PYTHONUNBUFFERED=1

WORKDIR /app

# System dependencies for OpenCV / YOLO
RUN apt-get update && apt-get install -y \
    libgl1 \
    libglu1-mesa \
    libglu1-mesa-dev \
    wget \
    && rm -rf /var/lib/apt/lists/*

# PyTorch CPU version (별도 설치)
RUN pip install --no-cache-dir torch torchvision torchaudio \
```

```
--index-url https://download.pytorch.org/whl/cpu

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port",
"8000"]
```

fe/Dockerfile

```
# Build stage
FROM node:20-alpine AS build
ARG VITE_API_BASE_URL=/api
ENV VITE_API_BASE_URL=$VITE_API_BASE_URL
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Production stage
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

8.6 UFW 방화벽 설정 (EC2)

```
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
```

```
sudo ufw enable
sudo ufw status
```

RabbitMQ 관리 UI(15672), Prometheus(9090), Grafana(3000)는 외부 직접 노출 없이 SSH 터널링으로 접근한다.

9. Nginx 설정

9.1 라우팅 구조

경로	대상
/	Frontend (React SPA)
/api/	Backend (Spring Boot 8081)

9.2 주요 설정

- SPA 라우팅: `try_files $uri $uri/ /index.html`
- SSL 인증서 경로: `/etc/letsencrypt/live/<DOMAIN>/`
- `proxy_read_timeout` 300s 설정 권장 (AI 비동기 처리 고려)

9.3 Nginx 설정 파일 (복사/붙여넣기)

infra/nginx/conf.d/default.conf

```
server {
    listen 80;
    server_name <DOMAIN>;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name <DOMAIN>;

    ssl_certificate      /etc/letsencrypt/live/<DOMAIN>/full
chain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/<DOMAIN>/priv
```

```

key.pem;

ssl_protocols TLSv1.2 TLSv1.3;
ssl_prefer_server_ciphers on;

client_max_body_size 20m;

location ~ /\. (git|env|svn|hg) {
    deny all;
    return 404;
}

location / {
    proxy_pass http://frontend:80;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwa
rded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

location /api/ {
    proxy_pass http://backend:8081/;
    proxy_connect_timeout 300s;
    proxy_send_timeout 300s;
    proxy_read_timeout 300s;
    send_timeout 300s;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwa
rded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header X-Forwarded-Prefix /api;
}
}

```

fe/nginx.conf (프론트엔드 컨테이너 내부 SPA 라우팅)

```

server {
    listen 80;

    location = /index.html {
        root    /usr/share/nginx/html;
        add_header Cache-Control "no-cache, no-store, must-revalidate";
        add_header Pragma "no-cache";
        add_header Expires 0;
    }

    location /assets/ {
        root    /usr/share/nginx/html;
        add_header Cache-Control "public, max-age=31536000, immutable";
    }

    location / {
        root    /usr/share/nginx/html;
        index   index.html index.htm;
        try_files $uri $uri/ /index.html;
    }
}

```

10. DB 접속 정보 및 ERD 연계

10.1 주요 DB 환경변수

변수명	설명
<code>DB_URL</code>	PostgreSQL JDBC URL (AWS RDS Endpoint 포함)
<code>DB_USERNAME</code>	DB 접속 계정
<code>DB_PASSWORD</code>	DB 접속 비밀번호

10.2 관련 설정 파일 목록

- `be/spring/src/main/resources/application.properties`
- `be/spring/src/main/resources/application-local.properties`

- `be/spring/src/main/resources/application-prod.properties`
- `infra/.env`

11. 외부 서비스 정보

서비스	용도	비고
Upstage Solar (GMS)	LLM 감정 해석	OpenAI 호환 API
AWS RDS	PostgreSQL DB	외부 관리형 DB
AWS S3	드로잉 이미지 저장	Presigned URL 방식
Let's Encrypt	SSL 인증서	Certbot 자동 갱신
RabbitMQ	AI 비동기 처리 큐	컨테이너 내 운영

12. 모니터링 가이드

모니터링 스택은 별도 Compose 파일로 분리되어 있다.

12.1 모니터링 서비스 기동

```
cd infra
docker compose -f docker-compose.monitoring.yml up -d
```

12.2 모니터링 구성

서비스	포트	용도
Prometheus	<code>127.0.0.1:9090</code>	메트릭 수집 (15일 보존)
Grafana	<code>127.0.0.1:3000</code>	대시보드 시각화
Loki	내부	로그 집계 (15일 보존)
Promtail	-	도커/Nginx 로그 수집
Node Exporter	-	시스템 메트릭
cAdvisor	-	컨테이너 메트릭
Alertmanager	-	Slack 알림 연동

Grafana, Prometheus, RabbitMQ 관리 UI는 SSH 터널링(127.0.0.1 바인딩)으로만 접근 가능하다.


```
# SSH 터널 예시
ssh -i <KEY.pem> \
  -L 3000:localhost:3000 \
  -L 9090:localhost:9090 \
  -L 15672:localhost:15672 \
  ubuntu@<EC2_HOST>
```

13. 빌드 및 배포 절차 요약

1. GitLab Repository Clone
2. 환경변수(`.env`) 설정 (`infra/.env` , `ai/.env`)
3. Docker Compose 이미지 빌드
4. Nginx Reverse Proxy / SSL 설정 (Certbot 자동 발급)
5. Backend / Frontend / AI / RabbitMQ / Redis 서비스 기동
6. 모니터링 스택 기동 (선택)

14. 테스트 체크리스트

항목	상태
API 연동 (Backend ↔ Frontend)	정상
Redis 연결	정상
RabbitMQ 메시지 큐	정상
AI 분석 콜백 (RabbitMQ → AI → Backend)	정상
드로잉 이미지 업로드 (S3)	정상
YOLO 객체 탐지	정상
HTTPS 접속	정상
OAuth 소셜 로그인	정상

15. 보안 관련 안내

보안상 민감한 정보(Upstage API Key, AWS 자격증명, DB 계정, RabbitMQ 계정)는 제출 문서에서 실제 값을 제외하였으며, 서버 환경에서는 `.env` 또는 CI/CD Secret으로 관리한다.

- `infra/.env` : 서버 환경에서만 작성, GitLab `.gitignore` 처리
- `ai/.env` : AI 서비스 자격증명, 동일하게 `.gitignore` 처리
- Grafana, Prometheus, RabbitMQ UI: SSH 터널링으로만 접근 (외부 포트 미노출)